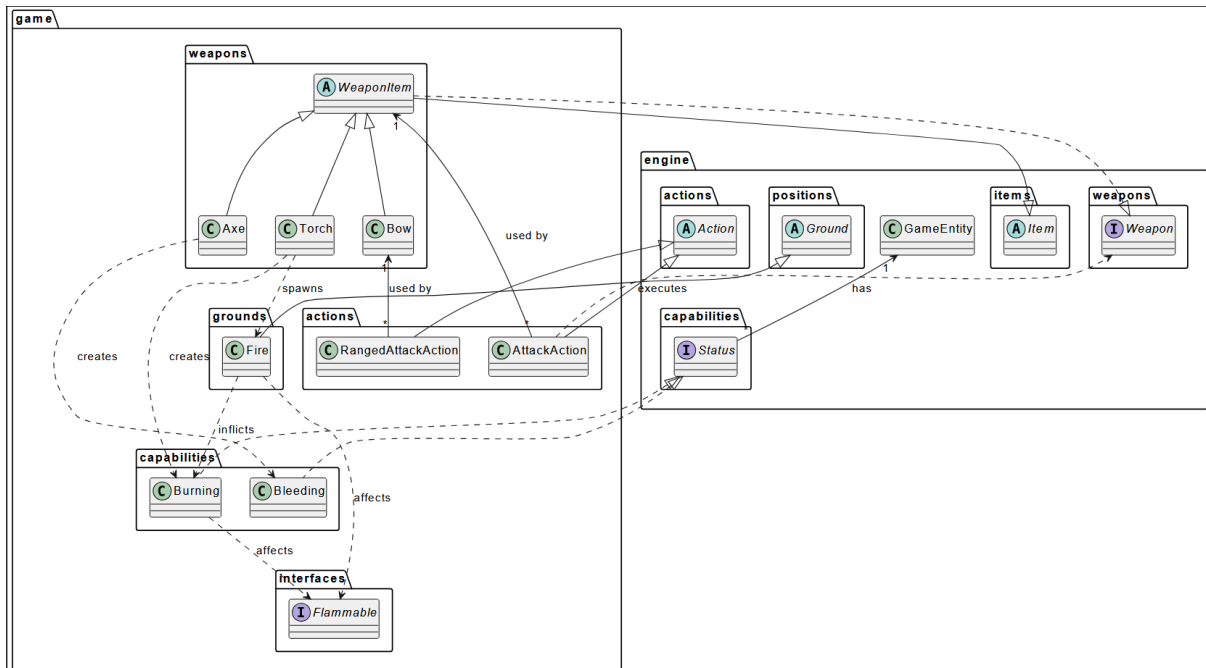
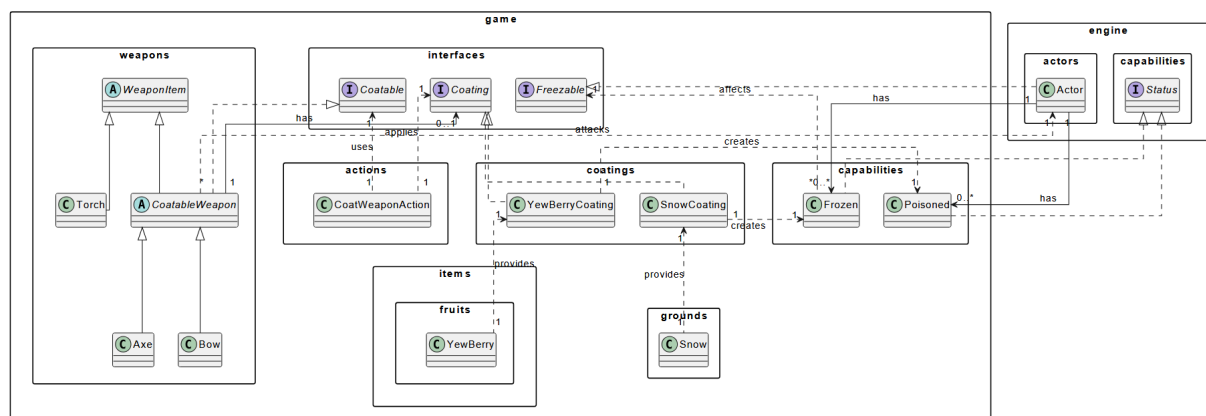


UML Diagram

REQ 3

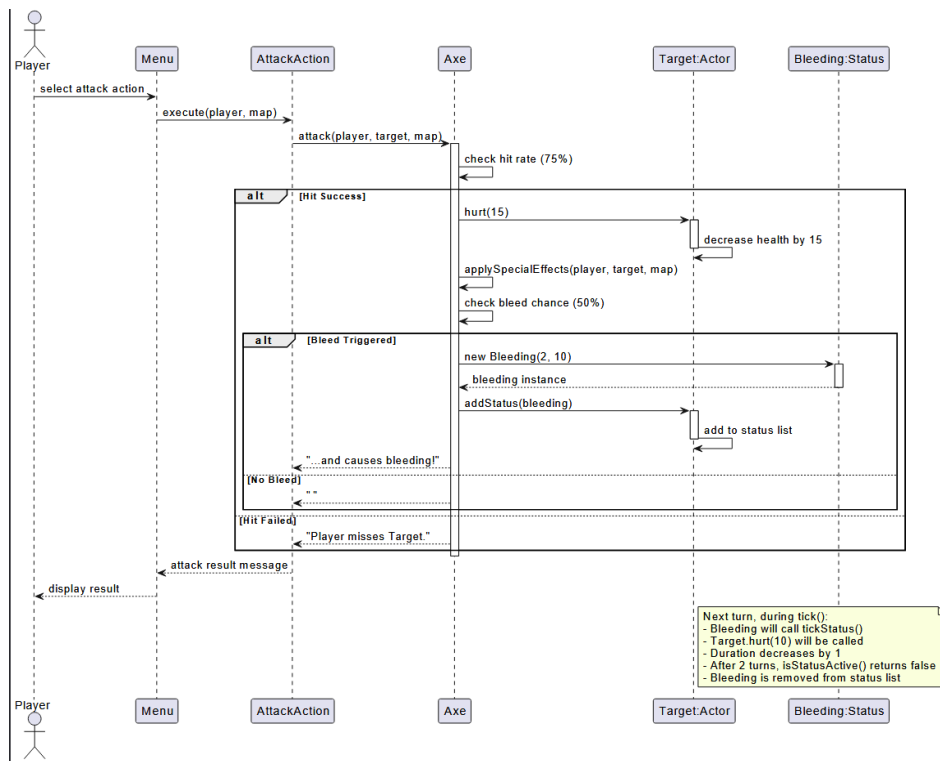


REQ 4

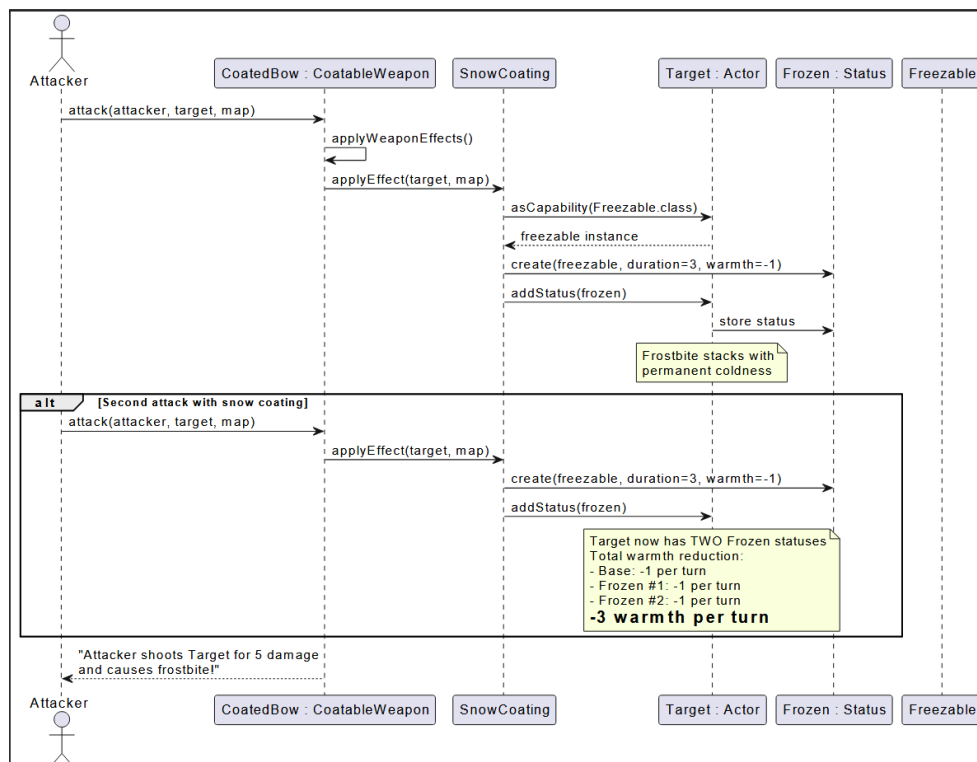


Sequence Diagram

REQ 3 Axe Attack with Bleeding Effect



REQ 4 Attacking with Snow Coating and Frostbite Stacking



Design Rationale for REQ3: Weapons System

Design Overview

The weapons system introduces three weapon types (Axe, Torch, Bow) with unique combat mechanics including status effects and environmental interactions. The design uses `WeaponItem` as an abstract base class, allowing weapons to implement unique special effects through polymorphic methods.

Design Structure

- **WeaponItem** as the abstract base class for all weapons
 - **Status effects** (Bleeding, Burning) implementing the Status interface
 - **Specialized actions** (RangedAttackAction) for ranged combat
 - **Environmental effects** (Fire ground) for area hazards
-

Alternative Design Considered

Weapon-Specific Actions and Behaviors

In this alternative approach, each weapon would be paired with its own custom attack action. For example, `AxeAttackAction` would encapsulate all axe combat logic (damage, hit rate, bleeding), while `TorchAttackAction` would handle torch attacks with burning and fire spawning.

Advantages:

- Clear encapsulation of weapon-specific logic in dedicated action classes
- Easy weapon identification by action type
- Weapon actions can be reused across different contexts

Disadvantages:

- **Code Duplication:** Hit rate checking, damage application, unconscious handling repeated across all weapon actions
- **Reduced Flexibility:** New weapons require creating both action and potentially behavior classes
- **Maintenance Overhead:** Global attack changes require updating every weapon action
- **Action Proliferation:** 10 weapons = 10+ new action classes

Chosen Design: WeaponItem Hierarchy

Advantages of the Design

Single Responsibility Principle (SRP)

- WeaponItem handles core mechanics (damage, hit rate, attack execution)
- Individual weapons (Axe, Torch, Bow) define only their unique effects
- Status effects encapsulate damage-over-time logic independently
- Fire manages environmental hazard behavior separately

Open-Closed Principle (OCP)

- New weapons added by extending WeaponItem without modifying existing classes
- New status effects added by implementing Status interface
- AttackAction works with any weapon polymorphically

Reusability Through Abstraction

- Weapon interface allows both IntrinsicWeapon and WeaponItem to share mechanics
- AttackAction works with any Weapon implementation
- Status effects apply to any GameEntity
- Common attack logic centralized in WeaponItem

Disadvantages of the Design

Code Duplication in Attack Logic

Each weapon repeats similar patterns for hit rate checking, damage application, and message formatting. While WeaponItem centralizes structure, weapons still share implementation patterns.

Tight Coupling to Status System

Weapons directly instantiate status effects, creating dependency on concrete Status implementations. A factory pattern could improve decoupling but adds complexity.

Range Attack Complexity

RangedAttackAction uses static helper method for target scanning, creating coupling between action generation and map scanning logic. A dedicated service class could improve modularity.

Connascence Analysis

The design maintains acceptable low-level connascence through interface abstractions (CoN, CoT) with well-defined method contracts, though it exhibits moderate Connascence of Algorithm (CoA) in duplicated hit rate checking across weapons and Connascence of Value (CoV) in status constructor parameters requiring coordinated changes. The chosen design

has lower overall connascence than weapon-specific actions, which would require higher Connascence of Algorithm across duplicated attack logic and increased Connascence of Execution coordinating weapon actions with their behaviors.

Conclusion

The weapons system demonstrates strong adherence to SOLID principles while maintaining manageable connascence levels. The use of the `WeaponItem` abstract class with polymorphic `applySpecialEffects()`, combined with the `Status` interface and concrete implementations (Bleeding, Burning), creates a flexible combat framework that:

1. Centralizes common attack logic in `WeaponItem`, avoiding duplication across weapon-specific action classes
2. Enables diverse weapon behaviors through polymorphic special effects without proliferating action classes
3. Supports stacking status effects for tactical depth in sustained combat
4. Separates attack modes (melee vs ranged) through specialized actions while maintaining unified weapon handling
5. Maintains low connascence through interface abstractions, allowing new weapons and status effects without modifying existing code

This design provides a robust foundation for the combat system that can accommodate new weapons, status effects, and attack mechanics without requiring structural changes to the codebase.

Design Rationale for REQ4: Coating

Design Overview

The coating system allows weapons (excluding torches) to be enhanced with special substances that apply additional effects when attacking. The design introduces a flexible framework where coatings can come from different sources (items or environmental grounds) and apply various status effects to targets.

Core Design Components

Coating Interface

A Coating interface serves as the contract for all coating types. This abstraction ensures that the system is open for extension - new coatings can be added without modifying existing code. Each coating implementation defines:

- The effect applied when a coated weapon hits a target
- A descriptive name for display purposes

Coatable Interface

The Coatable interface marks objects that can receive coatings. This design choice allows for future extensibility beyond just weapons (e.g., arrows, traps) without restructuring the hierarchy. The interface provides methods to:

- Apply a coating
- Check if currently coated
- Retrieve the current coating
- Remove a coating

CoatableWeapon Abstract Class

CoatableWeapon serves as a specialized weapon type that implements Coatable. This abstract class:

- Extends WeaponItem to inherit core weapon functionality
- Implements Coatable to gain coating capabilities
- Delegates weapon-specific effects to subclasses via `applyWeaponEffects()`
- Applies both weapon effects AND coating effects during attacks

This separation allows weapons like Axe and Bow to focus on their unique mechanics (bleeding, ranged attacks) while the coating system remains modular.

Torch Exclusion

Torch extends `WeaponItem` directly rather than `CoatableWeapon`, explicitly excluding it from coating mechanics. This design decision respects the requirement that torches cannot be coated, using the type system itself to enforce this constraint rather than runtime checks.

Status Effect System Integration

The coating system integrates seamlessly with the existing status effect framework:

Poisoned Status (YewBerry)

When a weapon coated with yewberry strikes a target, a Poisoned status is added with:

- Duration: 5 turns
- Damage: 4 per turn
- Stacking: Multiple poison applications accumulate

The Poisoned class implements the Status interface, handling its own tick logic and expiration.

Frozen Status (Snow)

Snow-coated weapons apply Frozen status, which:

- Duration: 3 turns
- Effect: Reduces warmth level by 1 per turn
- Requires the target to implement Freezable interface
- Stacks with permanent environmental coldness

The Freezable interface ensures type safety - only entities that can experience frostbite are affected. This follows the Interface Segregation Principle.

Coating Application System

CoatWeaponAction

`CoatWeaponAction` encapsulates the logic of applying a coating to a weapon. The action distinguishes between two coating sources:

1. **Item-based coatings** (e.g., YewBerry): The item is consumed after use
2. **Ground-based coatings** (e.g., Snow): The source remains available

This design allows for different acquisition mechanics while using the same action framework.

Contextual Availability

Coating actions are made available through `allowableActions()` based on context:

- **YewBerry**: Available when in the actor's inventory
- **Snow**: Available when the actor stands on snow ground

This ensures coatings are only accessible when logically appropriate.

Advantages of the Design

Single Responsibility Principle (SRP)

- Coating implementations only define their specific effects
- CoatableWeapon only manages coating state
- CoatWeaponAction only handles the coating application process
- Individual weapons (Axe, Bow) focus on their unique mechanics

Open-Closed Principle (OCP)

The design is open for extension but closed for modification:

- New coatings can be added by implementing Coating
- New coatable items can implement Coatable
- New status effects can implement Status
- No existing code needs modification when adding new coating types

Dependency Inversion Principle (DIP)

High-level modules depend on abstractions:

- CoatWeaponAction depends on Coatable interface, not concrete weapons
- Weapons depend on Coating interface, not concrete coating types
- Attack logic depends on Status interface, not specific status implementations

Interface Segregation Principle (ISP)

- Freezable interface ensures only appropriate entities can be frozen
- Coatable interface is focused solely on coating behavior
- Not all weapons must be coatable (Torch is excluded naturally)

Liskov Substitution Principle (LSP)

- Any Coating can be used interchangeably where coatings are expected
- Any CoatableWeapon can be coated regardless of its specific type
- Any Status implementation can be processed by the status system

Don't Repeat Yourself (DRY)

- Coating application logic is centralized in CoatWeaponAction
- Status effect ticking is handled uniformly by the engine's status system
- Attack logic with coating effects is implemented once in CoatableWeapon

Disadvantages and Trade-offs

Complexity for Simple Effects

For weapons with straightforward coating interactions, the abstraction layers (interfaces, abstract classes) may seem excessive. However, this upfront complexity pays dividends when extending the system.

State Management Overhead

Each coatable weapon must maintain its coating state and handle replacement logic. While this is managed in CoatableWeapon, it does add memory overhead per weapon instance..

Connassance Analysis

The design exhibits primarily static connassance forms (Name, Type, Position) which are manageable through interface contracts and compile-time checking—for example, all Coating implementations must agree on method names (applyEffect, getName) and return types. More complex connassance forms like Connassance of Algorithm (the status ticking mechanism must be understood by both Status implementations and the engine) are minimized by encapsulating behavior within individual status classes, keeping coupling loose and maintainable.

Alternative Design Considerations

Alternative 1: Coating as Item Attributes

Coatings could have been modeled as mutable attributes within weapon classes, with switch statements determining effects. This would be simpler initially but would:

- Violate OCP when adding new coatings
- Centralize coating logic in weapons rather than coating classes
- Make it harder to share coatings across different coatable types

The chosen design balances simplicity with extensibility, using interfaces and abstract classes to provide clear extension points without over-engineering.

Conclusion

The coating system demonstrates strong adherence to SOLID principles while maintaining practical simplicity. The use of the Coating interface with concrete implementations (YewBerryCoating, SnowCoating), combined with the Coatable interface and CoatableWeapon abstract class, creates a flexible framework that:

1. Clearly separates coating sources (items vs. ground) from coating effects
2. Integrates seamlessly with the existing status effect system

3. Naturally excludes inappropriate items (Torch) through type structure
4. Supports diverse acquisition mechanics through contextual actions
5. Enables easy addition of new coatings, coatable items, and status effects

This design provides a robust foundation for the coating mechanic that can evolve with future gameplay requirements without requiring structural changes to the codebase.